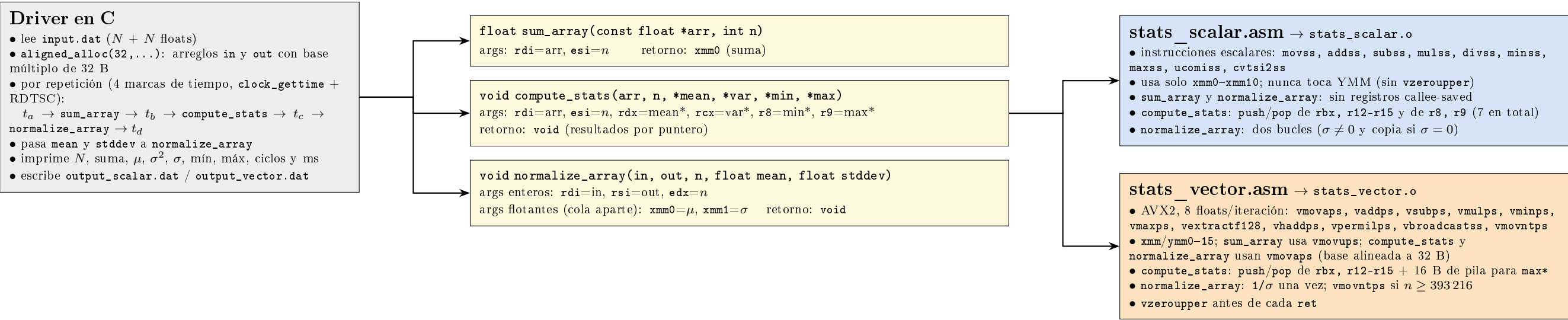


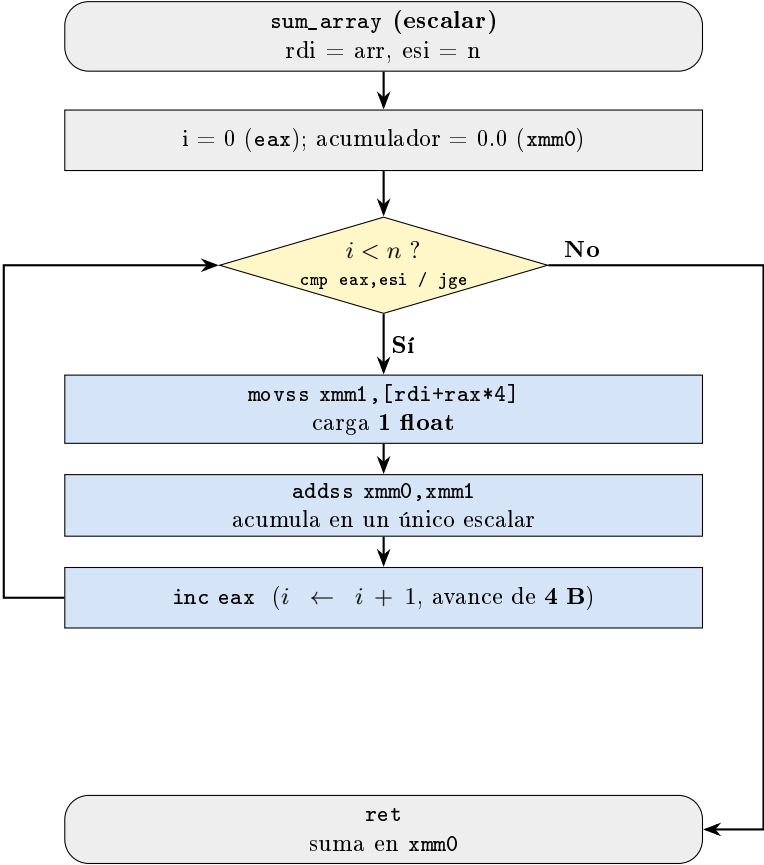
Arquitectura de software: driver C, kernels NASM y convención de llamada System V AMD64

INTERFAZ (idéntica en ambas versiones): llamadas `call` respetando la ABI System V

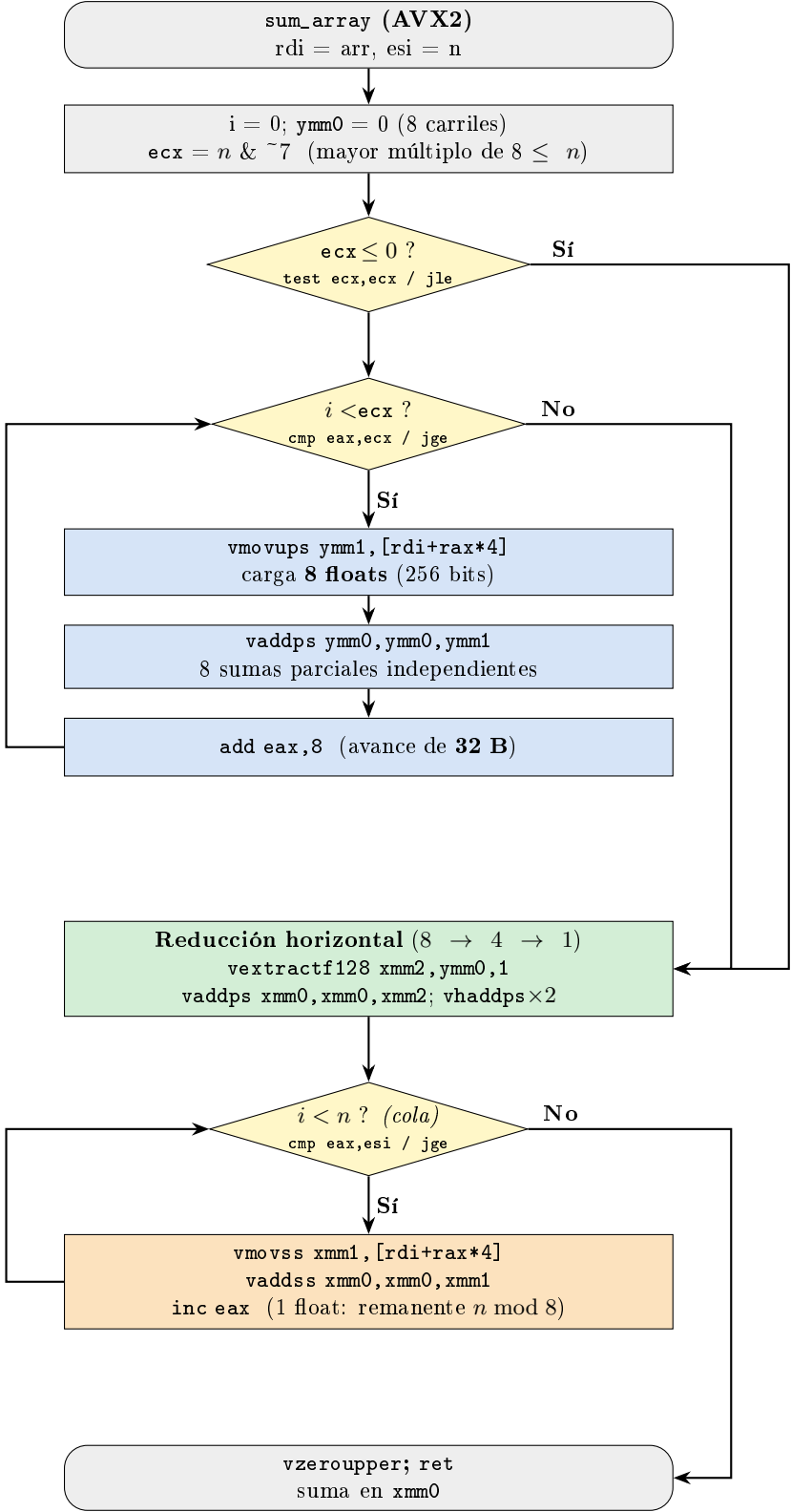


Flujo de control de sum_array: escalar (1 elemento/iteración) vs. vectorial AVX2 (8 elementos/iteración + cola)

VERSIÓN ESCALAR

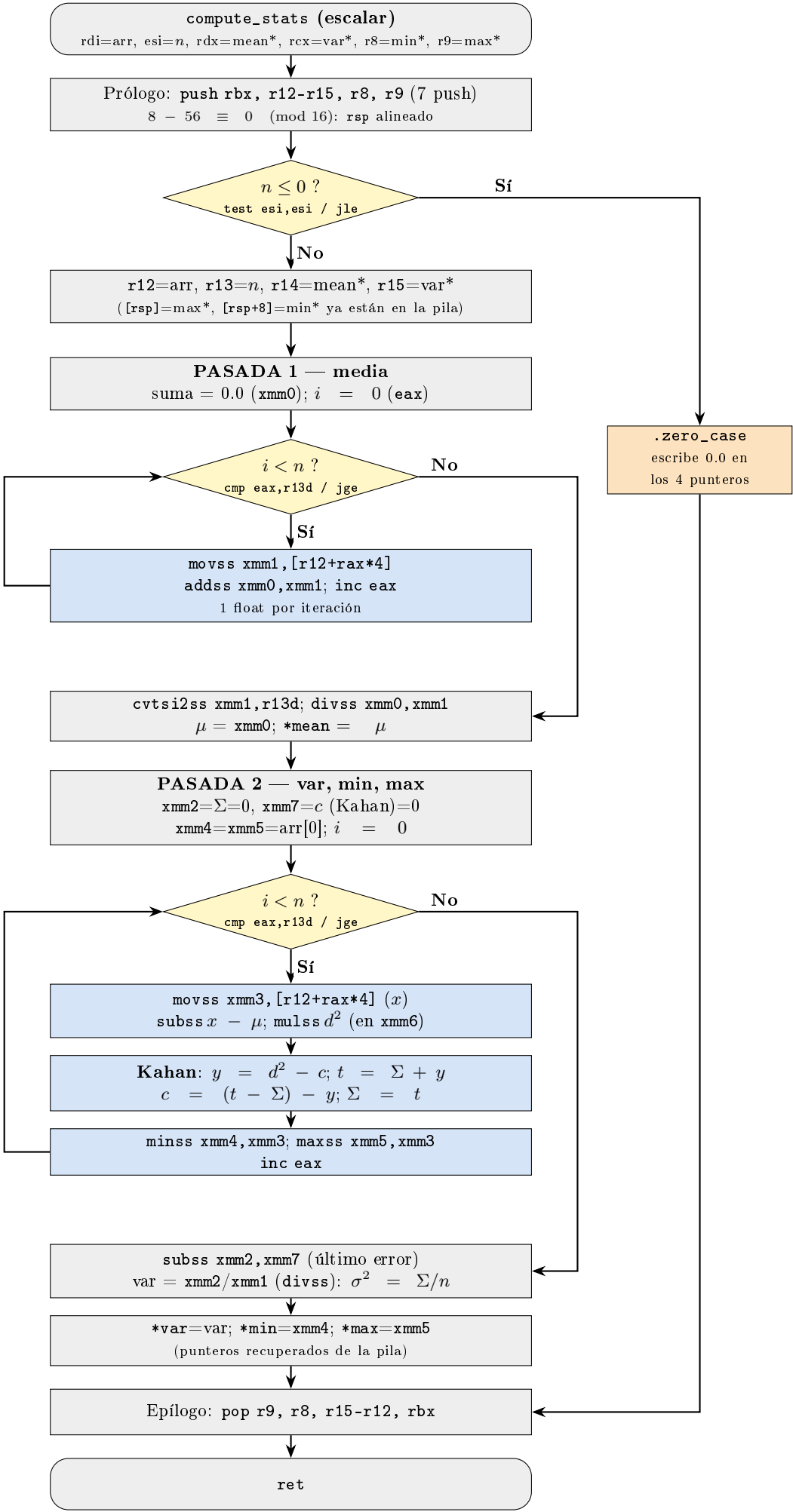


VERSIÓN VECTORIAL (AVX2)

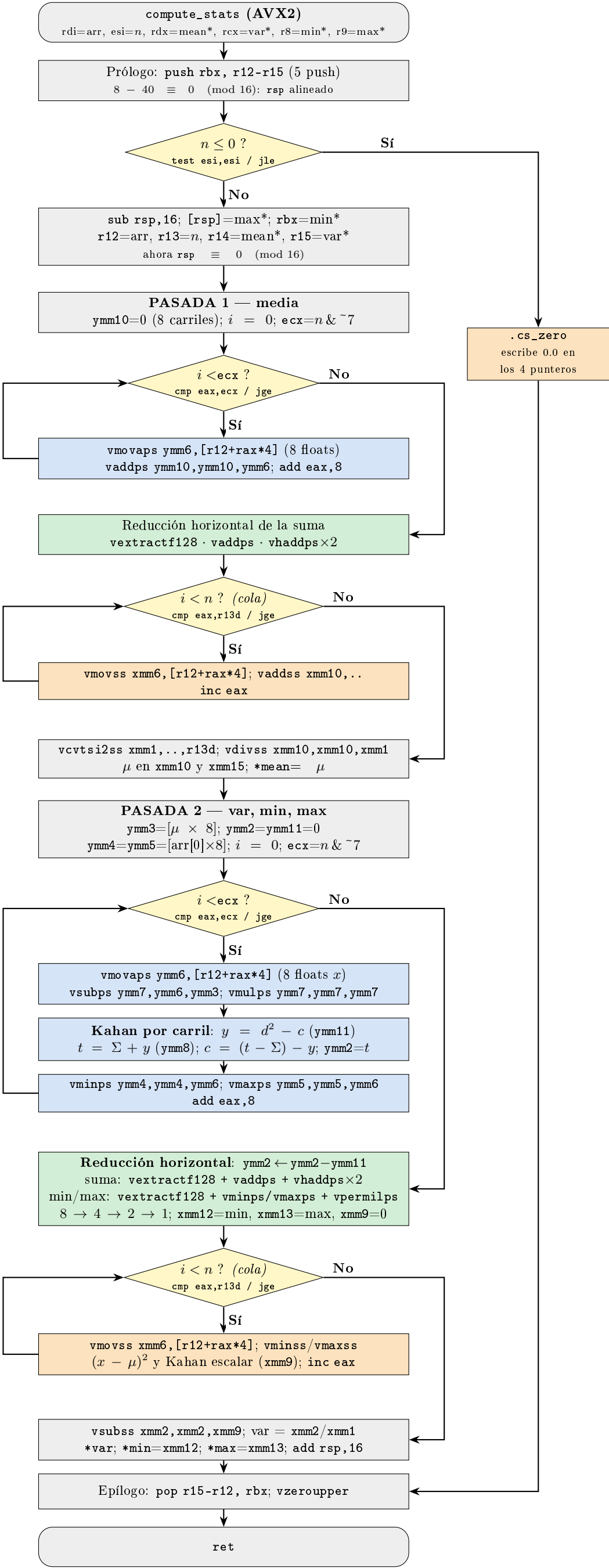


Flujo de control de compute_stats: escalar (dos pasadas, 1 elemento/iteración) vs. vectorial AVX2 (8 elementos/iteración + cola)

VERSIÓN ESCALAR

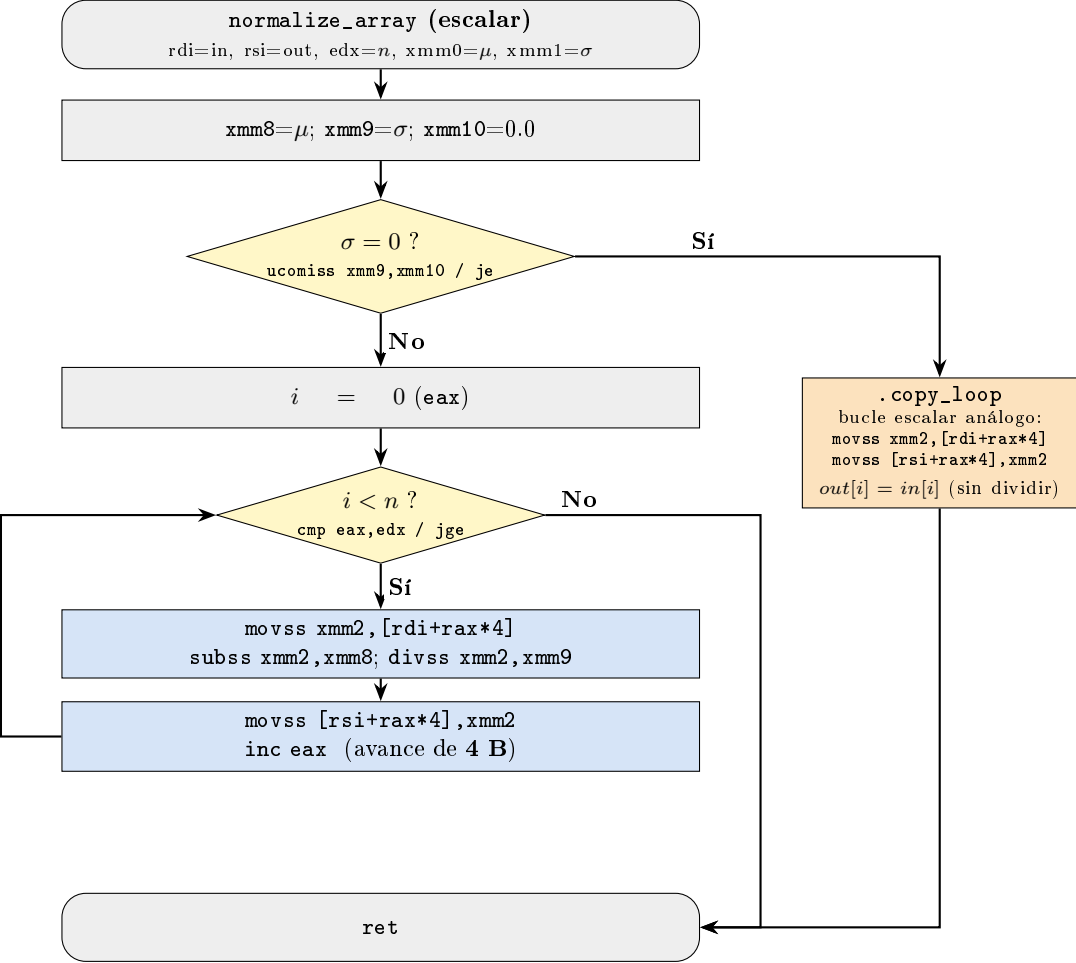


VERSIÓN VECTORIAL (AVX2)



Flujo de control de normalize_array: escalar vs. vectorial AVX2 (camino temporal / no temporal + cola)

VERSIÓN ESCALAR



VERSIÓN VECTORIAL (AVX2)

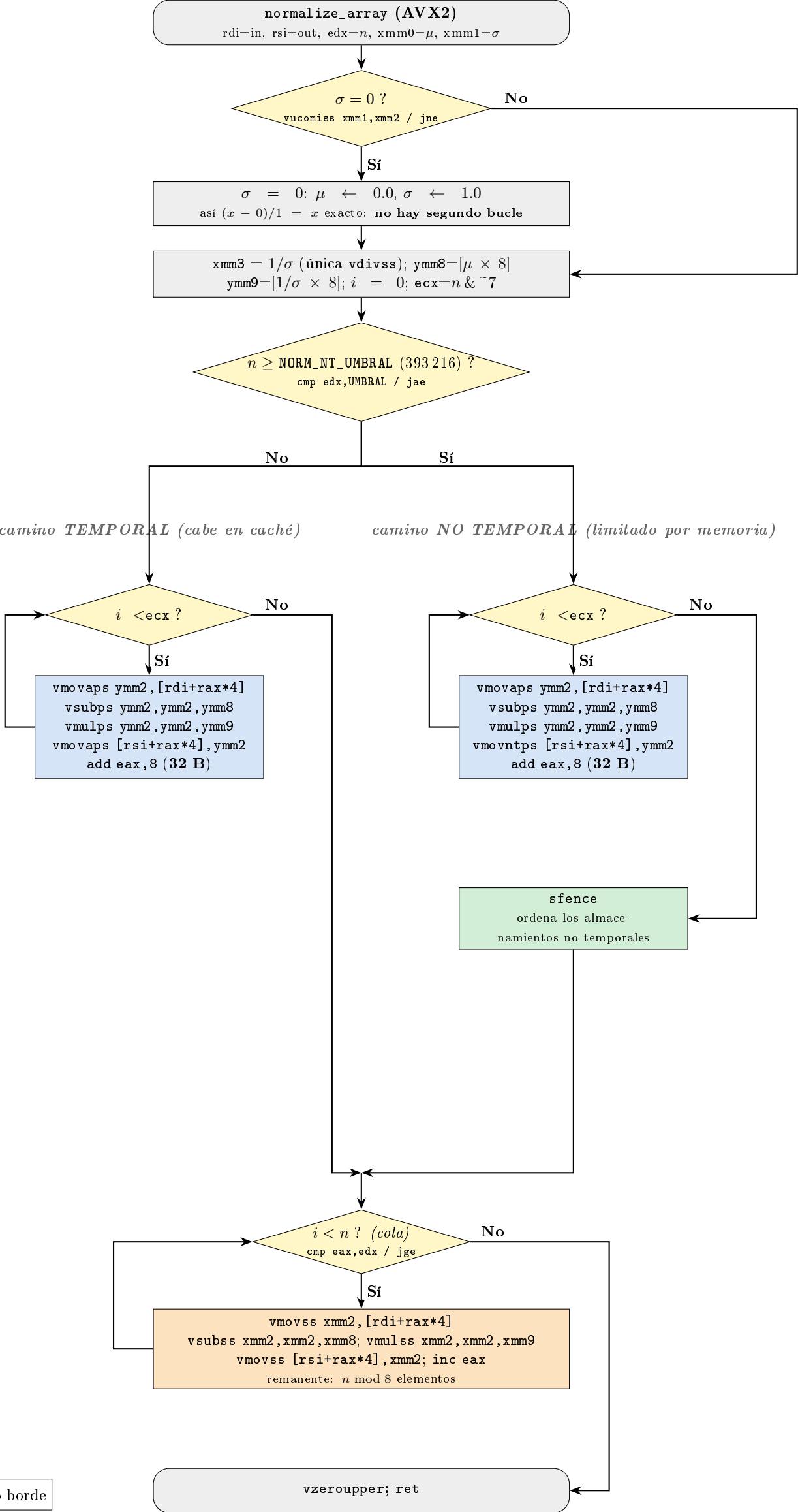


Tabla de asignación de registros por kernel (escalar vs. vectorial)

Los registros `xmmk` son la mitad baja (128 bits) de `ymmk`. “Carril” = una de las 8 posiciones de 32 bits de un registro YMM. Los argumentos entran según la ABI System V y el kernel no los reasigna hasta copiarlos.

`sum_array(arr, n)` — **retorno en xmm0**

Registro	Escalar	Vectorial (AVX2)
<code>rdi</code>	<code>arr</code> (puntero base)	<code>arr</code> (puntero base)
<code>esi</code>	<code>n</code>	<code>n</code> (límite de la cola)
<code>eax</code>	<code>i</code> (índice, avanza de 1 en 1)	<code>i</code> (avanza de 8 en 8 en el bucle y de 1 en 1 en la cola)
<code>ecx</code>	—	<code>n & ~7</code> : límite del bucle vectorial
<code>xmm0</code>	acumulador escalar; retorno	tras la reducción: suma de los 8 carriles; sigue acumulando la cola; retorno
<code>ymm0</code>	—	8 acumuladores parciales (uno por carril)
<code>xmm1/ymm1</code>	<code>xmm1 = arr[i]</code>	<code>ymm1 = 8 floats</code> cargados (bucle); <code>xmm1 = arr[i]</code> (cola)
<code>xmm2</code>	—	mitad alta de <code>ymm0</code> (carriles 4–7) durante la reducción

`compute_stats(arr, n, *mean, *var, *min, *max)` — **dos pasadas: media; luego varianza + mín + máx**

Registro	Escalar	Vectorial (AVX2)
<code>rdi,esi,rdx,rcx,r8,r9</code>	al entrar: <code>arr</code> , <code>n</code> , <code>mean*</code> , <code>var*</code> , <code>min*</code> , <code>max*</code> (se copian a registros callee-saved o a la pila)	
<code>r12</code>	<code>arr</code>	<code>arr</code>
<code>r13</code>	<code>n</code>	<code>n</code> (<code>r13d</code>)
<code>r14</code>	<code>mean*</code>	<code>mean*</code>
<code>r15</code>	<code>var*</code>	<code>var*</code>
<code>rbx</code>	preservado, sin uso	<code>min*</code>
<code>[rsp]</code>	<code>max*</code> (<code>r9</code> apilado)	<code>max*</code> (<code>sub rsp,16</code> ; <code>[rsp] ← r9</code>)
<code>[rsp+8]</code>	<code>min*</code> (<code>r8</code> apilado)	relleno de alineación
<code>eax</code>	<code>i</code> en ambas pasadas	<code>i</code> en ambas pasadas y en las colas
<code>ecx</code>	(destruido; al final recupera <code>min*</code>)	<code>n & ~7</code> (pasadas 1 y 2)
<code>xmm0</code>	pasada 1: suma → media μ (se conserva en la pasada 2)	—
<code>xmm1</code>	(float) <code>n</code>	(float) <code>n</code> (no se toca en los bucles)
<code>xmm2</code>	$\Sigma(x - \mu)^2$ (acumulador)	<code>ymm2 = Σ</code> por carril; tras la reducción, Σ total en <code>xmm2[0]</code>
<code>xmm3</code>	<code>arr[i]</code>	<code>ymm3 = μ</code> en los 8 carriles
<code>xmm4</code>	mínimo	<code>ymm4 =</code> mínimo por carril → mínimo global en <code>xmm4[0]</code>
<code>xmm5</code>	máximo	<code>ymm5 =</code> máximo por carril → máximo global en <code>xmm5[0]</code>
<code>xmm6</code>	$d = x - \mu$, luego d^2	<code>ymm6 = 8 floats</code> cargados (en la cola: <code>xmm6 = arr[i]</code>)
<code>xmm7</code>	<code>c</code> (compensación de Kahan)	<code>ymm7 = (x - μ)²</code> y término corregido <code>y</code> (cola: <code>t</code>)
<code>xmm8</code>	$t = \Sigma + y$ (temporal)	<code>ymm8 = t</code> por carril; <code>xmm8 =</code> mitad alta / permutación en las reducciones
<code>xmm9</code>	—	compensación de Kahan de la cola
<code>ymm10</code>	—	pasada 1: 8 acumuladores de la suma → <code>xmm10 = suma</code> → μ
<code>ymm11</code>	—	compensación de Kahan por carril
<code>xmm12, xmm13</code>	—	mínimo y máximo escalares (la cola continúa desde la reducción)
<code>xmm15</code>	—	μ escalar (la cola lo necesita)

`normalize_array(in, out, n, mean, stddev)`

Registro	Escalar	Vectorial (AVX2)
<code>rdi</code>	<code>in</code>	<code>in</code> (alineado a 32 B)
<code>rsi</code>	<code>out</code>	<code>out</code> (alineado a 32 B)
<code>edx</code>	<code>n</code>	<code>n</code>
<code>eax</code>	<code>i</code> (de 1 en 1)	<code>i</code> (de 8 en 8 y de 1 en 1 en la cola); antes del bucle guarda 1.0 (0x3F800000)
<code>ecx</code>	—	<code>n & ~7</code>
<code>xmm0</code>	μ (argumento)	μ (argumento; se pone a 0.0 si $\sigma = 0$)
<code>xmm1</code>	σ (argumento)	σ (argumento; se pone a 1.0 si $\sigma = 0$)
<code>xmm2</code>	<code>in[i]</code> y resultado temporal	0.0 para comparar σ ; luego <code>ymm2 = 8 floats</code> (carga → resta → producto → almacenamiento)
<code>xmm3</code>	—	$1/\sigma$ (una sola <code>vdivss</code>)
<code>xmm8 / ymm8</code>	μ (copia)	μ en los 8 carriles (en la cola se usa <code>xmm8</code>)
<code>xmm9 / ymm9</code>	σ (copia)	$1/\sigma$ en los 8 carriles (en la cola se usa <code>xmm9</code>)
<code>xmm10</code>	0.0 (para <code>ucomiss</code>)	—

Disposición de memoria y avance del puntero (ejemplo con $N = 20$: $N \& \sim 7 = 16$)

Arreglo de entrada `in[]` (`rdi`): base múltiplo de 32 B (`aligned_alloc(32,...)`), 4 B por float

línea de caché (64 B) = 16 floats (si la base es múltiplo de 64)

siguiente línea

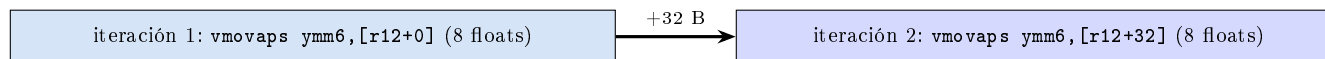
índice i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
<code>arr[i]</code>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
offset (B)	+0	+4	+8	+12	+16	+20	+24	+28	+32	+36	+40	+44	+48	+52	+56	+60	+64	+68	+72	+76

Versión escalar: 1 elemento por iteración, dirección = `rdi + rax*4`, `rax = 0, 1, 2, 3, ...`



cada flecha = avance de 4 B por iteración; n iteraciones en total

Versión vectorial: 8 elementos (32 B) por iteración, `rax = 0, 8, 16, ...` (`add eax,8`)



`vmovaps` exige dirección múltiplo de 32 B (si no, *fault*): se cumple porque la base es múltiplo de 32 y el avance es de 32 B

cola: `rax=16..19`, +4 B c/u

Carril del registro YMM que procesa cada elemento (`vaddps ymm0, ymm0, ymm1` acumula por carril)

carril 0	carril 1	carril 2	carril 3	carril 4	carril 5	carril 6	carril 7	carril 0	carril 1	carril 2	carril 3	carril 4	carril 5	carril 6	carril 7	escalar	escalar	escalar	escalar
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	----------	---------	---------	---------	---------

el carril k acumula `arr[k]`, `arr[k+8]`, `arr[k+16]`,... de forma independiente $\Rightarrow$ 8 sumas parciales que la reducción horizontal combina al final

Arreglo de salida `out[]` (`rsi`): también múltiplo de 32 B, misma disposición

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
vmovaps/vmovntps [rsi+0] (32 B)								[rsi+32] (32 B)								vmovss [rsi+rax*4], 4 B c/u			

Evidencia en GDB ($N = 16$): `rdi = 0x55555555b4c0` y `rsi = 0x55555555a480`; ambos $\equiv 0 \pmod{32}$ (`rsi mod 32 = 0`).

Escalar: offset = $4i$ bytes. Vectorial: offset = $32j$ bytes por iteración j en el bucle principal y $4i$ bytes en la cola.

— frontera de 32 B — frontera de línea de caché (64 B): cada línea contiene dos cargas YMM completas, así que ninguna carga vectorial cruza líneas.